

Nível de Microarquitetura (parte 2)



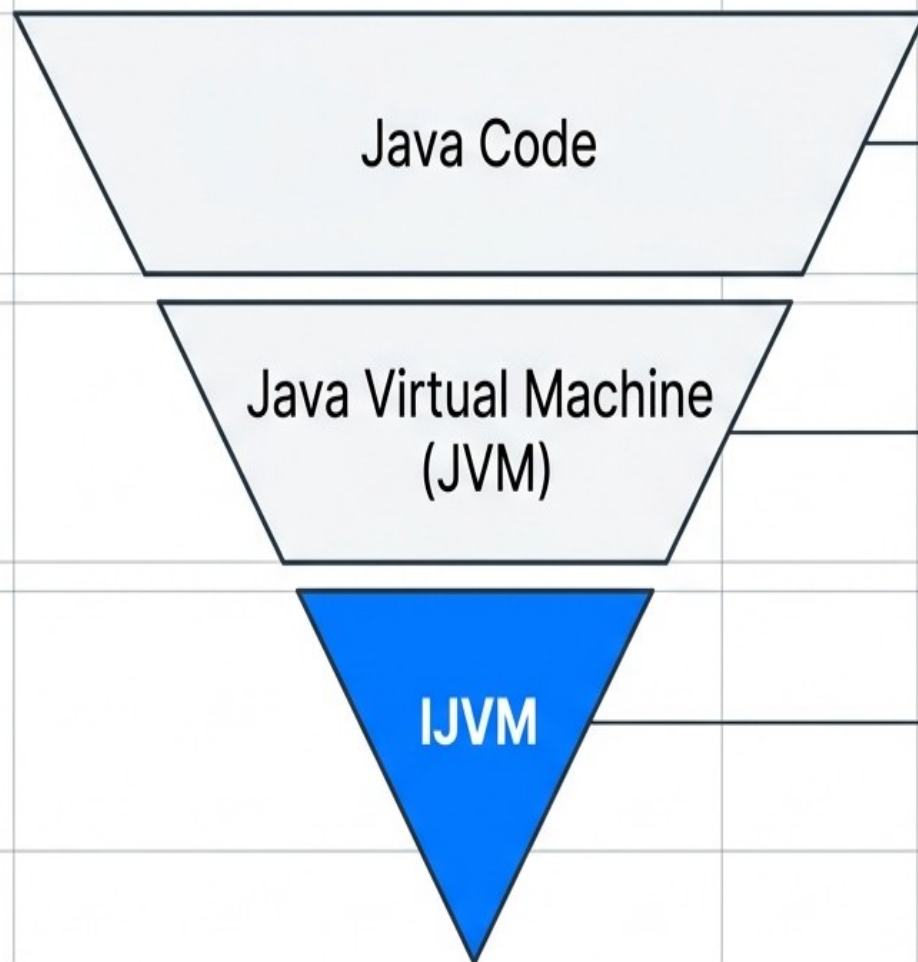
UNIFEI

Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

Prof. Carlos Minoru Tamaki

O que é a IJVM? (Contexto e Propósito)



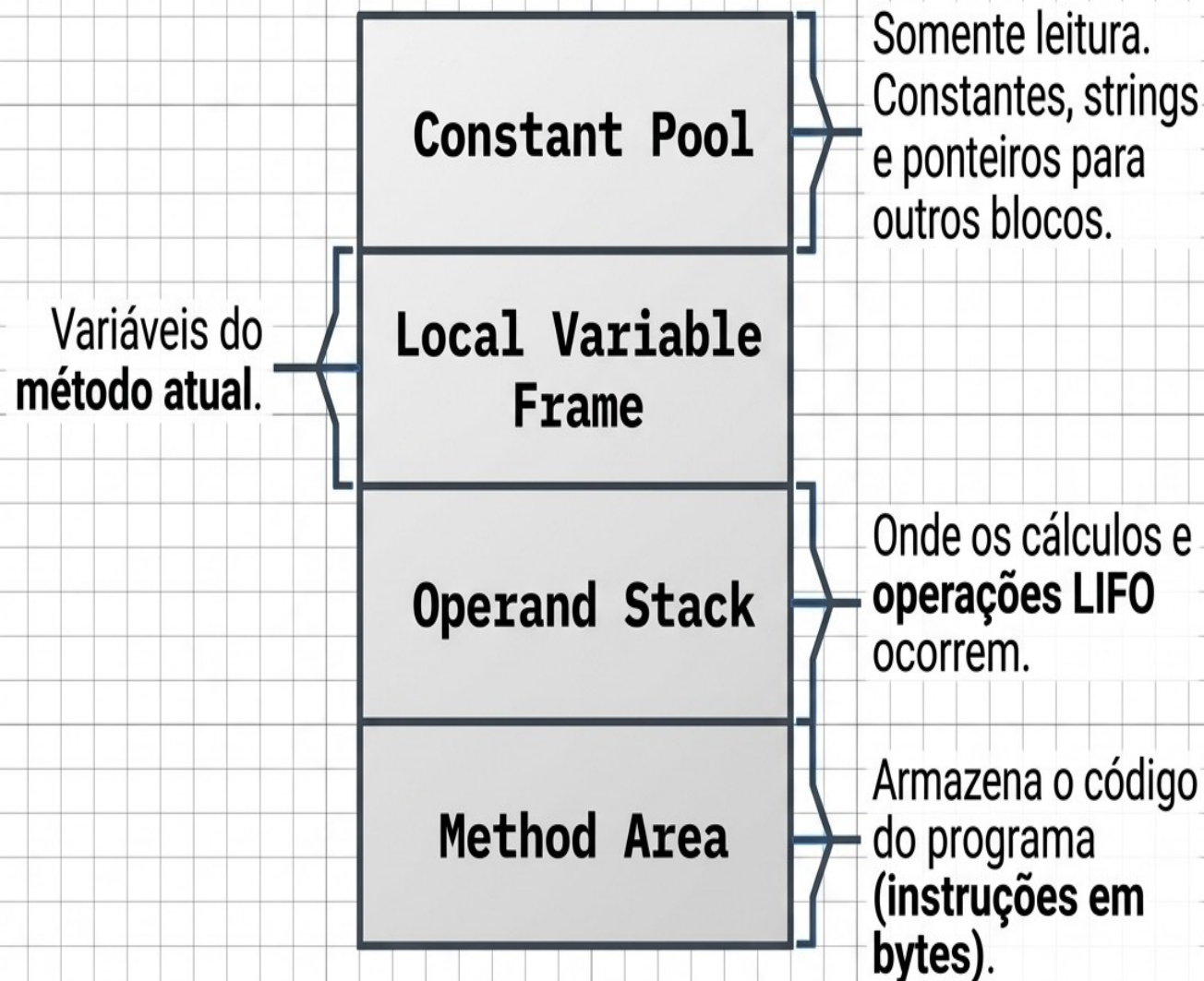
Linguagem de alto nível,
independente de plataforma.

Arquitetura complexa com
múltiplos tipos de dados.

Subconjunto educacional
simplificado.

Foco exclusivo em números inteiros (Integer).
Omitindo ponto flutuante para revelar os
princípios fundamentais da microarquitetura.

O Modelo de Memória da JVM (Visão Geral)



Modelo de memória JVM

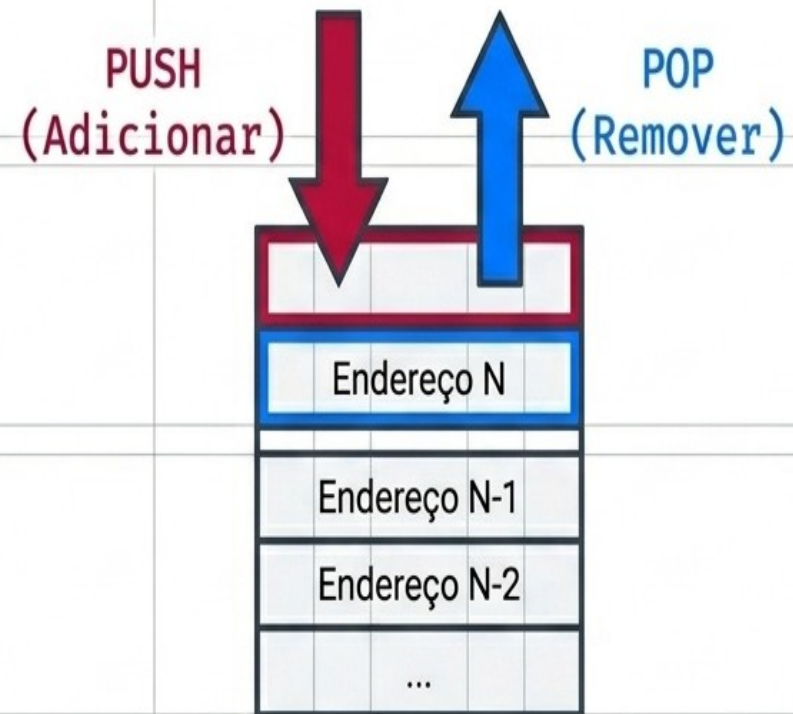
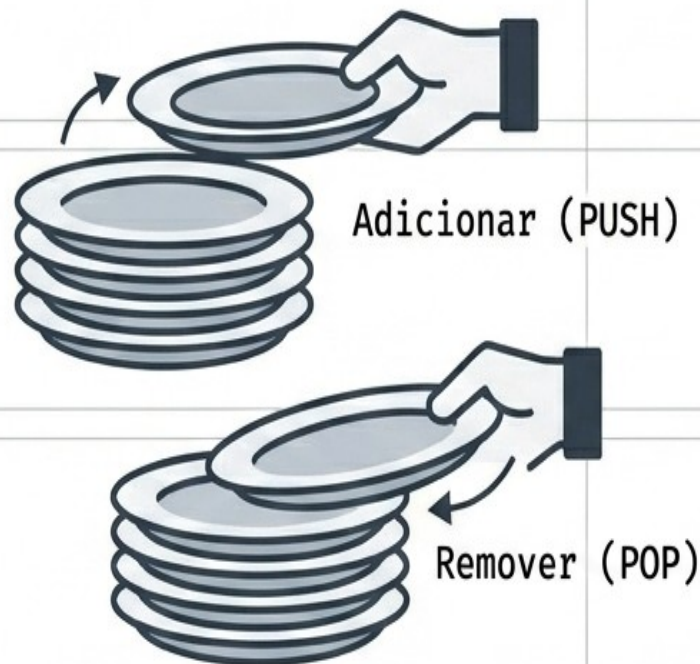
- As seguintes áreas de memória são definidas:
 1. O conjunto de constantes: esta área não pode ser escrita por um programa JVM e consiste em constantes, cadeias e ponteiros para outras áreas de memória. O registrador que controla esta área é o **CPP** que contém o endereço da primeira palavra do conjunto de constantes.
 2. Quadro de variáveis locais: para cada invocação de um método é alocado uma área para armazenar variáveis durante o tempo de invocação, denominado quadro de variáveis locais. Não inclui pilha de operandos. O registrador que controla essa área é o **LV**.

Modelo de memória JVM

3. A pilha de operandos: é garantido que o quadro não exceda um determinado tamanho, calculado antecipadamente pelo compilador Java. O espaço da pilha de operandos é colocado diretamente acima do quadro de variáveis. O registrador que controla o topo da pilha é o **SP**.
4. Área de método: há uma região de memória que contém o programa. O registrador que controla essa área é o **PC**.

O Paradigma de Pilha (Mecanismo LIFO)

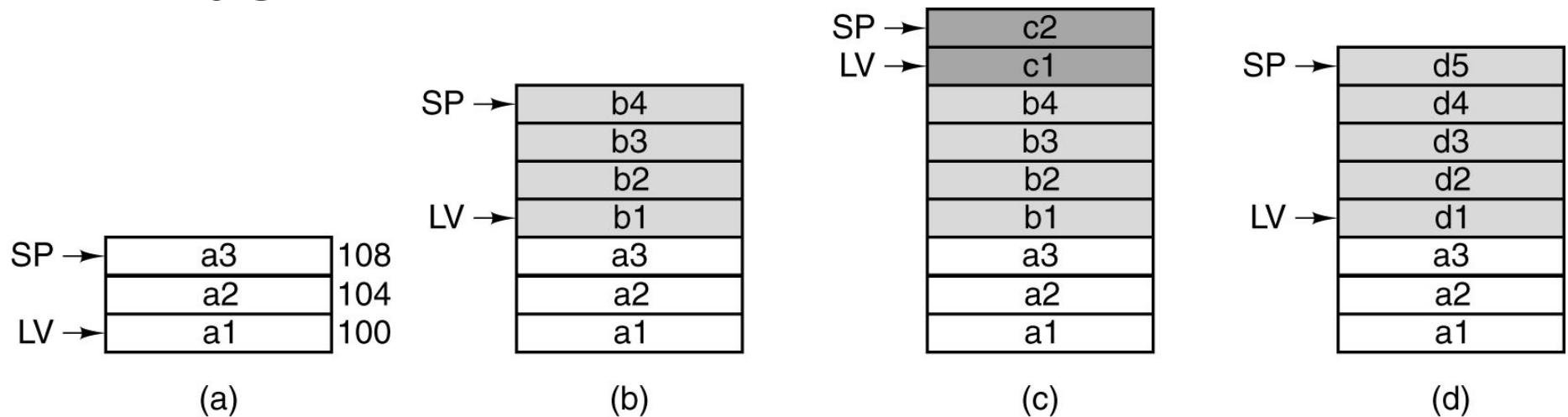
LIFO (Last-In, First-Out) - O último elemento a entrar é o primeiro a sair.



- A JVM não possui registradores gerais para armazenar operandos.
- Toda a matemática e lógica ocorre no Topo da Pilha.

Exemplo de ISA: IJVM

- Pilhas**



Uso de uma pilha para armazenar variáveis locais.

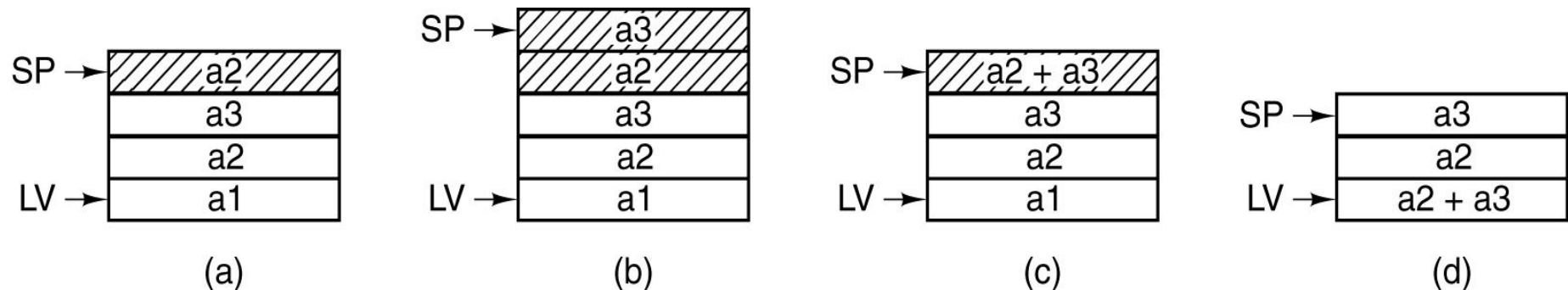
a) Enquanto A está ativo. c) Após B chamar C.

b) Após A chamar B.

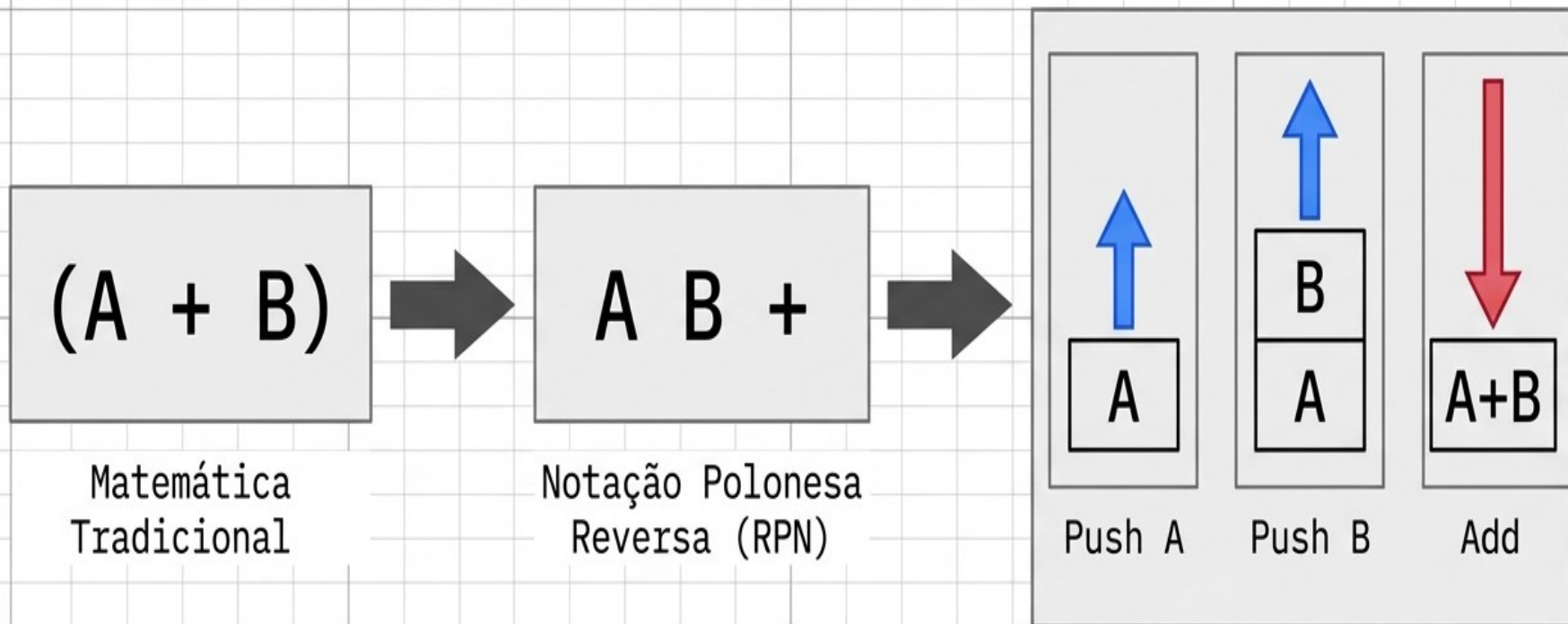
d) Após C e B retornarem e A chamar D.

Exemplo de ISA: IJVM

Utilização de uma pilha de operandos para efetuar um cálculo aritmético.

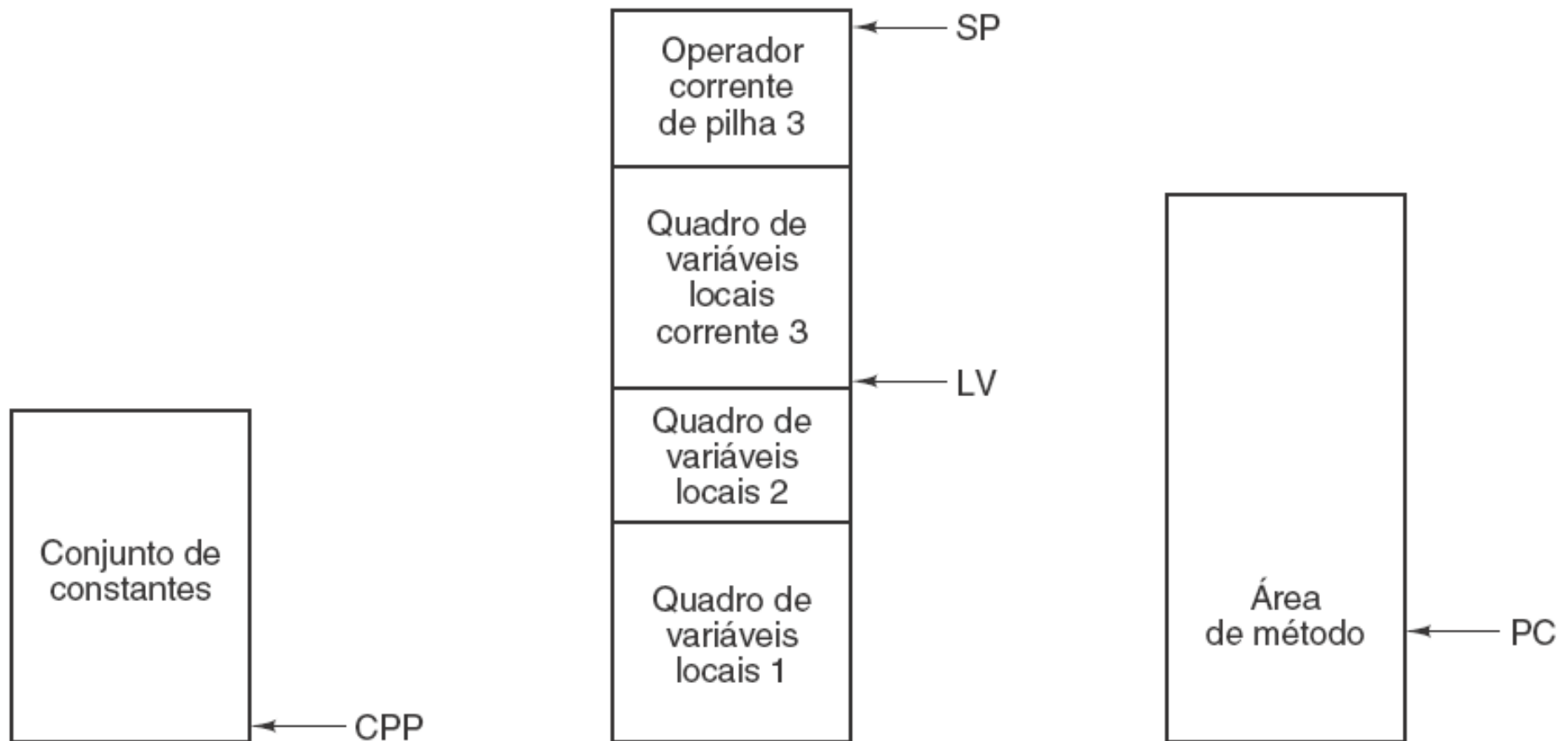


Por que usar Pilhas? (Notação Polonesa Reversa)



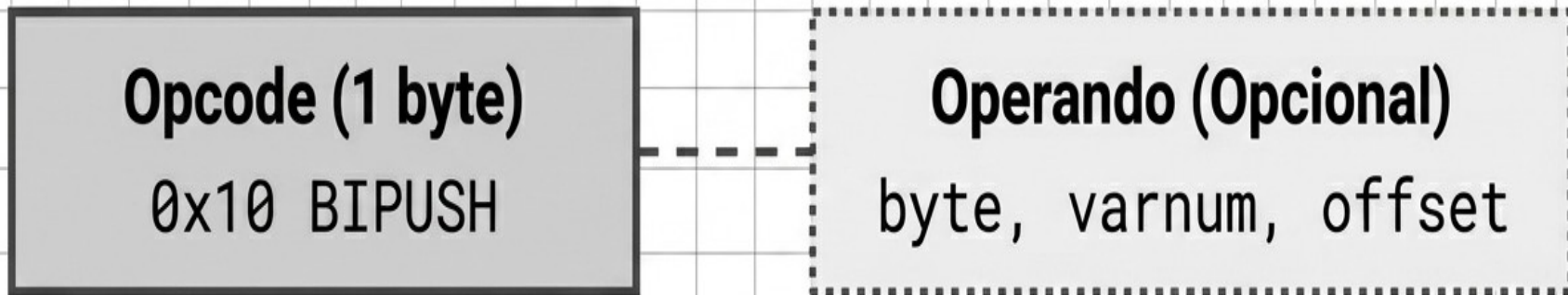
Nota: Na IJVM, os operandos devem ser carregados (PUSH) antes que o operador seja chamado. O operador consome os itens do topo e devolve o resultado.

As várias partes da memória JVM



O Conjunto de Instruções JVM (Visão Geral)

A JVM possui um conjunto reduzido de instruções (ISAs) projetado para manipular a pilha e os ponteiros.



- Cada instrução inicia com um Opcode hexadecimal de 1 byte.
- Algumas instruções requerem parâmetros adicionais (operandos).
- Avançam o PC de acordo com o tamanho da instrução.

O conjunto de instruções IJVM

Hex	Mnemônico	Significado
0x10	BIPUSH byte	Carregue o byte para a pilha
0x59	DUP	Copie a palavra do topo da pilha e passe-a para a pilha
0xA7	GOTO offset	Desvio incondicional
0x60	IADD	Retire duas palavras da pilha; carregue sua soma
0x7E	IAND	Retire duas palavras da pilha; carregue AND booleano
0x99	IFEQ offset	Retire palavra da pilha e desvie se for zero
0x9B	IFLT offset	Retire palavra da pilha e desvie se for menor do que zero
0x9F	IF_ICMPEQ offset	Retire duas palavras da pilha; desvie se iguais
0x84	IINC varnum const	Some uma constante a uma variável local
0x15	ILOAD varnum	Carregue variável local para pilha
0xB6	INVOKEVIRTUAL disp	Invoke um método
0x80	IOR	Retire duas palavras da pilha; carregue OR booleana
0xAC	IRETURN	Retorne do método com valor inteiro
0x36	ISTORE varnum	Retire palavra da pilha e armazene em variável local
0x64	ISUB	Retire duas palavras da pilha; carregue sua diferença
0x13	LDC_W index	Carregue constante do conjunto de constantes para pilha
0x00	NOP	Não faça nada
0x57	POP	Apague palavra no topo da pilha
0x5F	SWAP	Troque as duas palavras do topo da pilha uma pela outra
0xC4	WIDE	Instrução prefixada; instrução seguinte tem um índice de 16 bits

O conjunto de instruções IJVM. Os operandos *byte*, *const*, e *varnum* são 1 byte. Os operandos *disp*, *index*, e *offset* são 2 bytes.

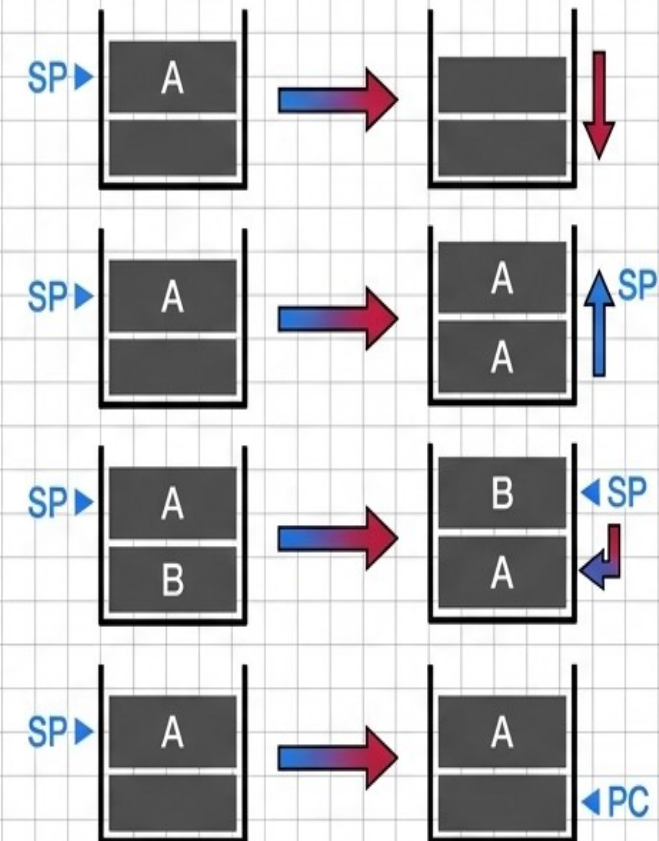
Matriz de Instruções: Movimentação de Dados

Opcode	Mnemônico	Ação	Efeito na Pilha (Antes -> Depois)
0x10	BIPUSH byte	Carrega o byte para o topo da pilha. 	... -> ..., byte
0x15	ILOAD varnum	Carrega variável local (LV + varnum) para a pilha. 	... -> ..., valor
0x36	ISTORE varnum	Retira do topo da pilha e armazena em variável local. 	..., valor -> ...
0x13	LDC_W index	Carrega constante (CPP + index) para a pilha. 	... -> ..., constante

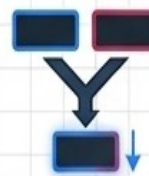
Matriz de Instruções: Manipulação de Topo

Mnemônico	Ação
POP (0x57)	Remove a palavra no topo da pilha.
DUP (0x59)	Copia a palavra do topo e a insere novamente.
SWAP (0x5F)	Troca as duas palavras do topo da pilha.
NOP (0x00)	Não faz nada (Avança PC).

Before & After

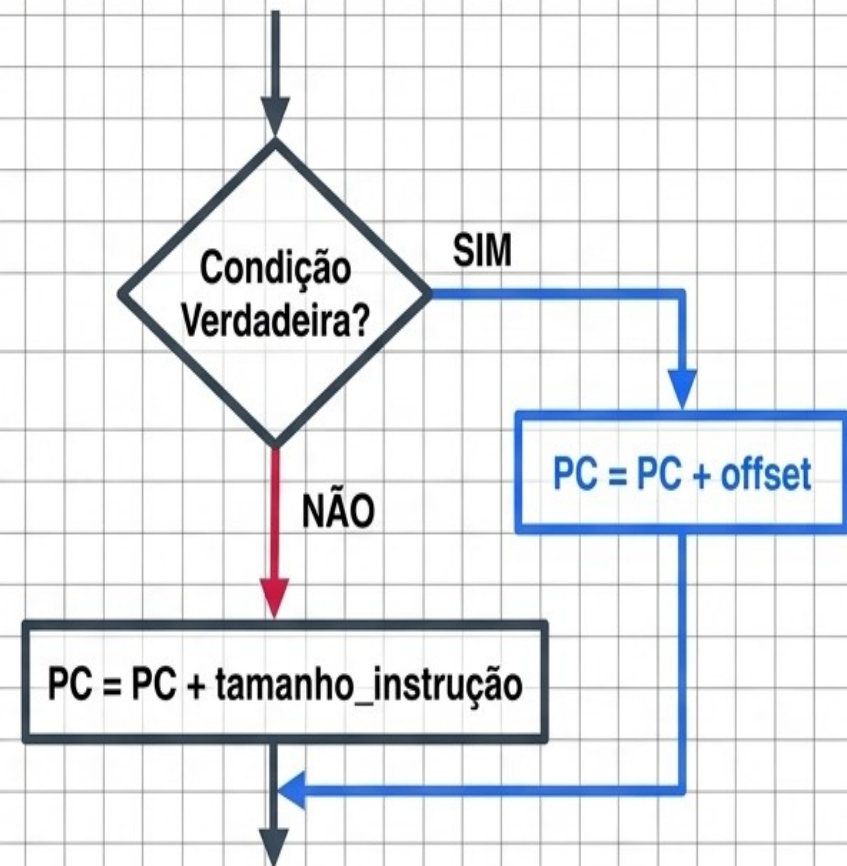


Matriz de Instruções: Aritmética e Lógica

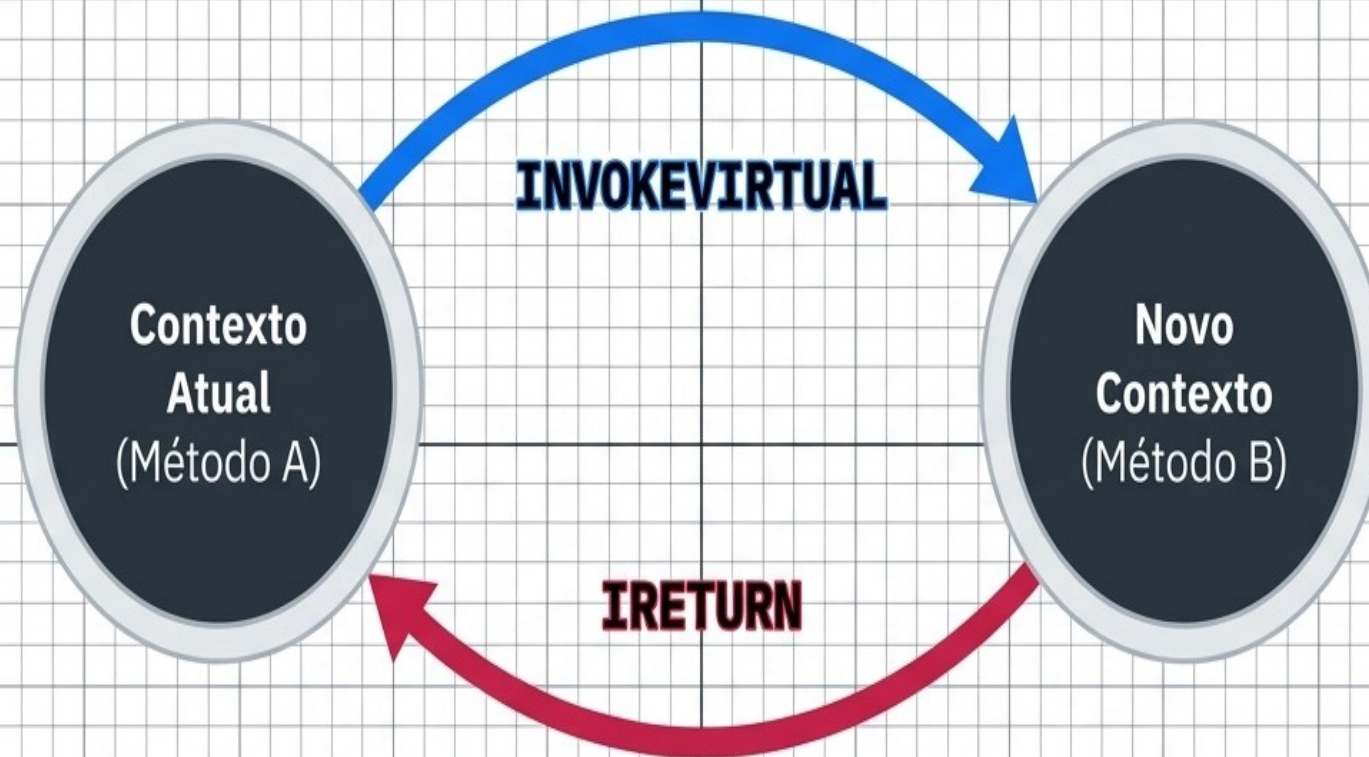
Mnemônico	Operação	Consumo LIFO
IADD (0x60)	Retira duas palavras da pilha; carrega sua soma.	
ISUB (0x64)	Retira duas palavras da pilha; carrega sua diferença.	
IAND (0x7E)	Retira duas palavras; carrega o resultado de AND booleano.	
IOR (0x80)	Retira duas palavras; carrega o resultado de OR booleano.	
IINC varnum const (0x84)	Soma uma constante diretamente a uma variável local (Não usa a pilha).	

Matriz de Instruções: Desvios (Control Flow)

Mnemônico	Ação
GOTO offset (0xA7)	Desvio incondicional.
IFEQ offset (0x99)	Retira o topo; desvia se for igual a zero.
IFLT offset (0x9B)	Retira o topo; desvia se for menor que zero.
IF_ICMPEQ offset (0x9F)	Retira dois do topo; desvia se forem iguais.



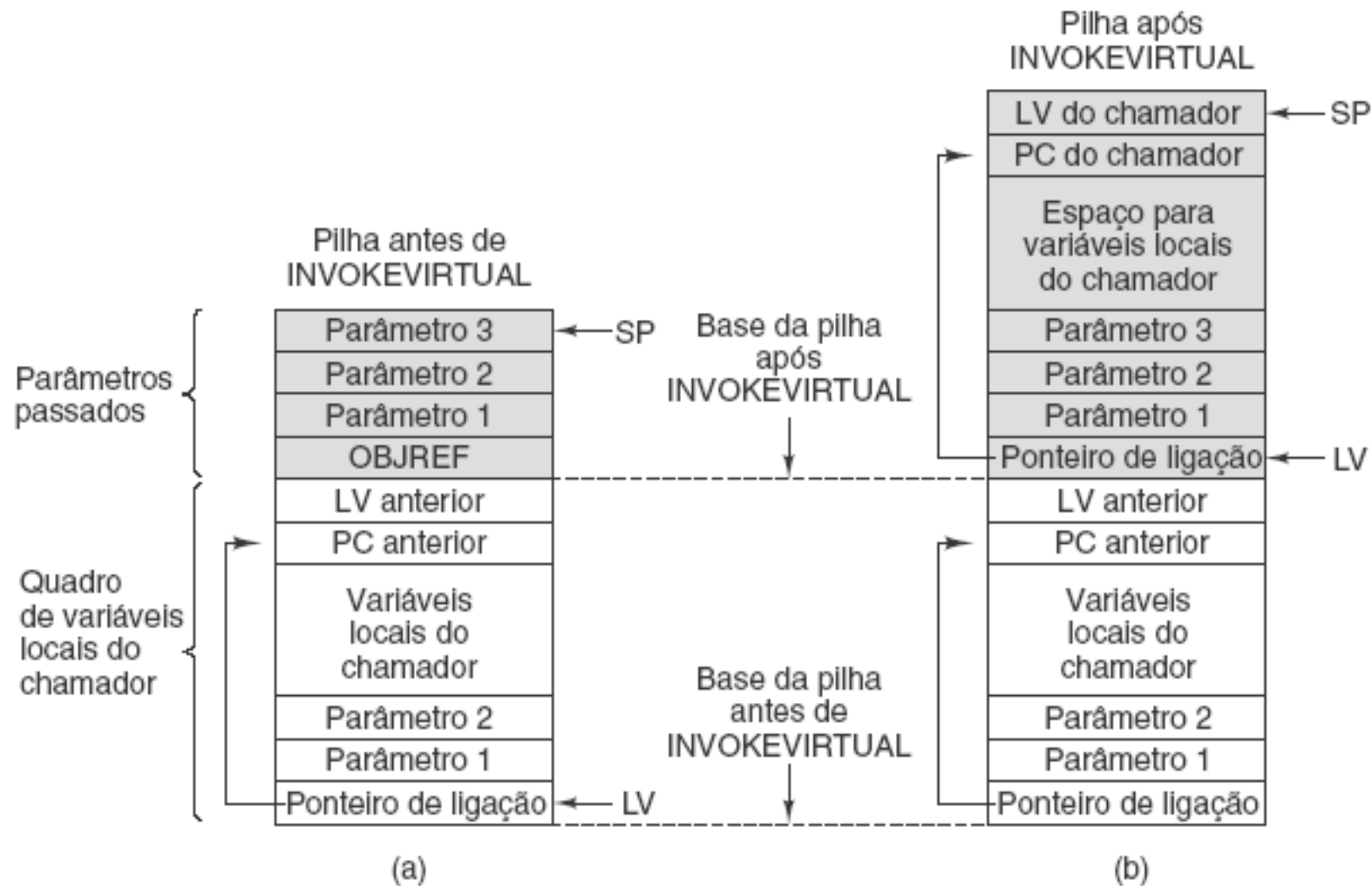
Invocação e Retorno de Métodos



INVOKEVIRTUAL (0xB6): Invoca um novo método. Complexo mecanismo que salva o estado atual e ajusta PC, LV e SP para o novo quadro.

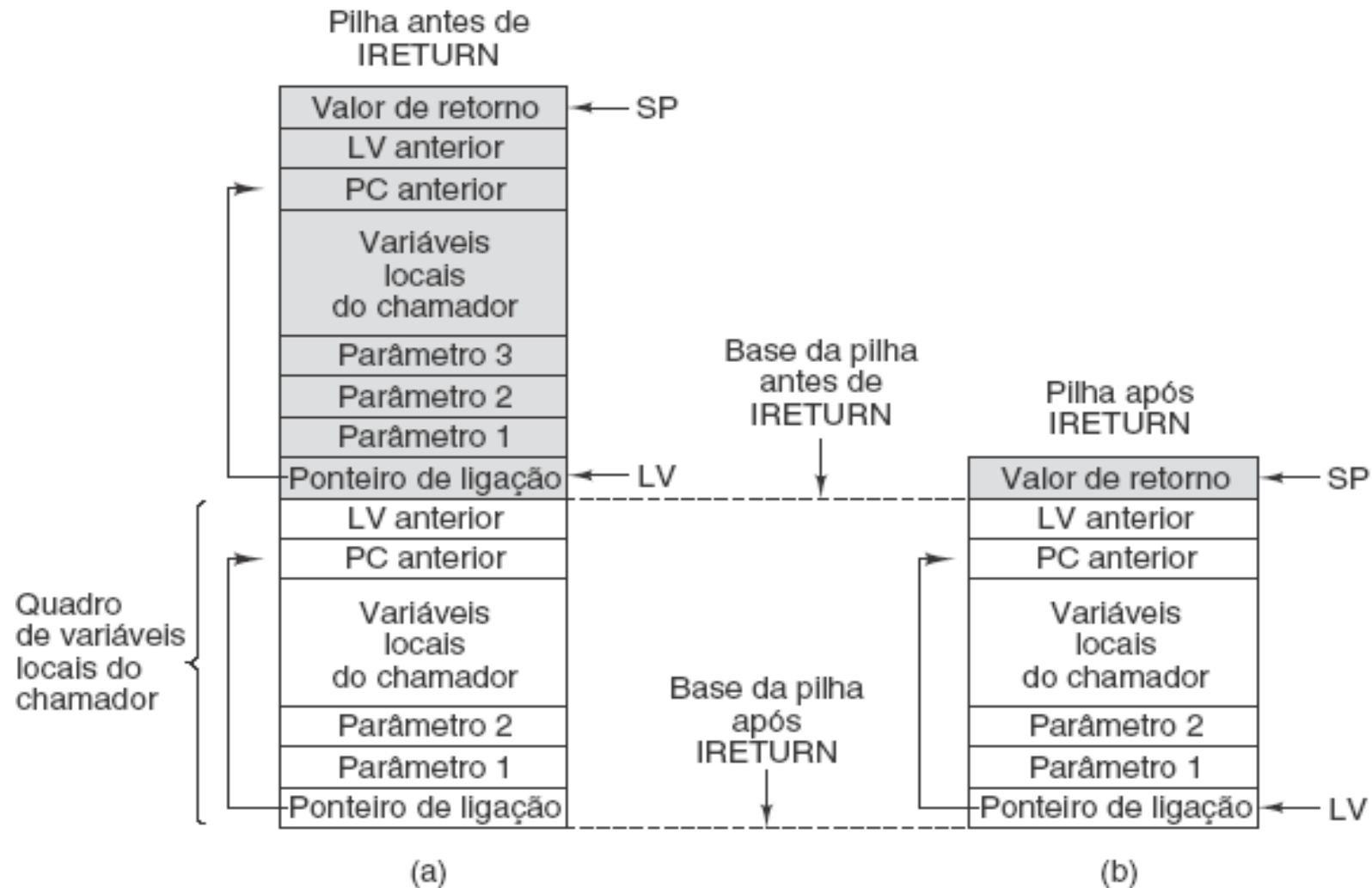
IRETURN (0xAC): Encerra o método. Restaura o estado salvo e coloca o resultado no topo da pilha do método chamador.

Instrução INVOKEVIRTUAL



- a) Memória antes de executar INVOKEVIRTUAL.
b) Após executá-la.

Instrução IRETURN



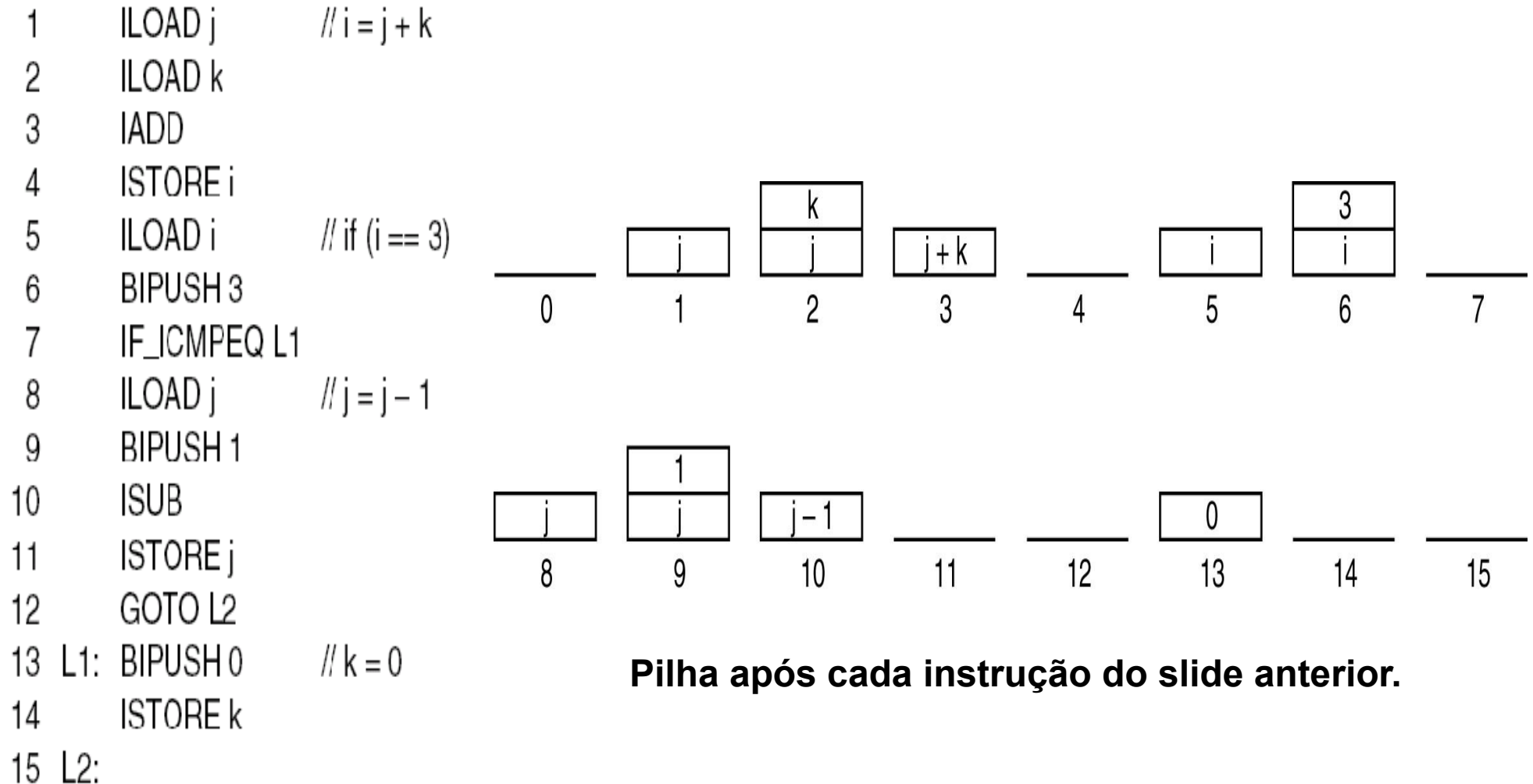
- a) Memória antes de executar IRETURN.
- b) Após executá-la.

Compilando Java para IJVM

i = j + k;	1	ILOAD j	// i = j + k	0x15 0x02
if (i == 3)	2	ILOAD k		0x15 0x03
k = 0;	3	IADD		0x60
else	4	ISTORE i		0x36 0x01
j = j - 1;	5	ILOAD i	// if (i == 3)	0x15 0x01
	6	BIPUSH 3		0x10 0x03
	7	IF_ICMPEQ L1		0x9F 0x00 0x0D
	8	ILOAD j	// j = j - 1	0x15 0x02
	9	BIPUSH 1		0x10 0x01
	10	ISUB		0x64
	11	ISTORE j		0x36 0x02
	12	GOTO L2		0xA7 0x00 0x07
	13 L1:	BIPUSH 0	// k = 0	0x10 0x00
	14	ISTORE k		0x36 0x03
	15 L2:			
(a)	(b)	(c)		

- a) **Fragmento de código Java.**
- b) **Linguagem de montagem Java correspondente.**
- c) **Programa IJVM em hexadecimal.**

Execução do programa IJVM



(b)

Bibliografia

- Organização Estruturada de Computadores – Andrew S. Tanenbaum, Tood Austin – 6ª Edição 2013 Prentice Hall